B2 SÉCURITÉ PRATIQUE

Dossier technique

Détection d’intrusions avec Wazuh

Déni de service HTTP · Injection SQL · Balayage de ports

Étudiante	Amal LABIDI
Module	B2 Sécurité Pratique — TP1 / TP2
Plateforme	Wazuh 4.14 (manager + dashboard), Debian
Techniques MITRE	T1499 · T1190 · T1046
Année	2026-2027
Date	Novembre 2026

1. Contexte et objectif

À partir de la plateforme Wazuh installée lors des TP1 et TP2, l'objectif est de mettre en œuvre **trois détections d'intrusion** : configurer l'agent et/ou le serveur, déclencher l'attaque depuis une machine de test, puis analyser l'alerte dans le dashboard. Une **réponse active** est ajoutée lorsqu'elle est pertinente (DoS).

#	Détection	MITRE ATT&CK	Source des logs	Règles	Réponse active
1	Déni de service HTTP	T1499 Endpoint DoS	Apache access.log	100200 → 100201 (perso)	Oui (firewall-drop)
2	Injection SQL	T1190 Exploit Public-Facing App.	Apache access.log	31103 / 31106 (natives)	Non
3	Balayage de ports Nmap	T1046 Network Service Discovery	iptables → kern.log	100300 → 100301 (perso)	Non

Cadre légal — Tous les tests sont réalisés **uniquement sur mes machines de laboratoire**, dans un **réseau isolé**. Accéder ou se maintenir frauduleusement dans un système de traitement automatisé de données est un délit (**art. 323-1 du Code pénal** : 3 ans d'emprisonnement et 100 000 € d'amende). Entraver son fonctionnement, comme le fait un DoS, est puni par l'art. 323-2. Ces outils (ab, nmap, curl) ne doivent jamais viser une cible sans autorisation écrite.

2. Schéma d'architecture

Trois machines virtuelles Debian sont reliées sur le même réseau privé isolé `192.168.213.0/24` (sans accès Internet).

Machine	Rôle	Adresse IP	Logiciels
Serveur Wazuh	Manager + indexer + dashboard : analyse, règles, alertes	192.168.213.157	wazuh-manager, dashboard
Agent « client1 » (ID 003)	Victime : serveur web, collecte des logs	192.168.213.134	Apache 2.4, wazuh-agent 4.14.9, iptables, rsyslog
Attaquante	Machine de test	192.168.213.159	ab (ApacheBench), curl, nmap 7.93

Flux simplifié

```
[ Attaquante ]  ----- HTTP / scan ----->  [ Agent client1 (Apache) ]
192.168.213.159                            192.168.213.134
                                                 |  logs (1514/tcp)
                                                 v
                                       [ Serveur Wazuh + Dashboard ]
                                           192.168.213.157
```

[Capture à insérer]

Schéma réseau (3 VM, IP, flux d'attaque et remontée des logs vers le manager)

3. Préparation du lab

- **Snapshots** des trois VM avant chaque test : retour rapide à un état propre en cas d'erreur de configuration.
- **Réseau isolé** (host-only) : aucune attaque ne peut sortir du lab. Un NAT temporaire a seulement servi à installer les paquets (voir §8).
- **Vérification de l'agent** depuis le serveur : l'agent doit être « Active ».

```
# Sur le serveur Wazuh
/var/ossec/bin/agent_control -l
#   ID: 003, Name: client1, IP: 192.168.213.134, Active

# Savoir si l'on est sur le serveur ou sur l'agent
/var/ossec/bin/wazuh-control info | grep type
```

Règle d'or — L'agent collecte, le serveur décide. Les blocs `<localfile>` (quels fichiers lire) se configurent dans `/var/ossec/etc/ossec.conf` de l'**agent**. Les règles (`local_rules.xml`) et la réponse active se configurent sur le **serveur**.

Avant chaque redémarrage du manager : `/var/ossec/bin/wazuh-analysisd -t` (test de syntaxe), puis `wazuh-logtest` pour valider une règle sur une ligne de log.

4. Détection 1 — Déni de service HTTP (MITRE T1499)

4.1 Explication de l'attaque

Un déni de service applicatif consiste à envoyer un très grand nombre de requêtes HTTP pour saturer le serveur web et le rendre indisponible (MITRE **T1499 — Endpoint Denial of Service**, sous-technique T1499.002 Service Exhaustion Flood). Chaque requête isolée est légitime : c'est la **fréquence** depuis une même IP qui révèle l'attaque. Wazuh ne la détecte pas nativement : il faut une règle de corrélation.

4.2 Procédure pas à pas

Étape 1 — Agent — faire lire le journal Apache à l'agent, puis `systemctl restart wazuh-agent` :

Agent — /var/ossec/etc/ossec.conf

```
<localfile>
  <log_format>apache</log_format>
  <location>/var/log/apache2/access.log</location>
</localfile>
```

Étape 2 — Serveur — ajouter deux règles : 100200 « marque » chaque requête simple, 100201 se déclenche quand 100200 est vue 100 fois en 10 s depuis la même IP source :

Serveur — /var/ossec/etc/rules/local_rules.xml

```
<group name="local,web,dos,">
  <rule id="100200" level="3">
    <if_sid>31108</if_sid>
    <description>Requete web recue (base flood)</description>
  </rule>

  <rule id="100201" level="10" frequency="100" timeframe="10">
    <if_matched_sid>100200</if_matched_sid>
    <same_source_ip />
    <description>DoS HTTP : plus de 100 requetes en 10 s depuis $(srcip)</description>
    <mitre>
      <id>T1499</id>
    </mitre>
  </rule>
</group>
```

Étape 3 — Serveur — réponse active : bloquer l'IP source sur l'agent pendant 180 s quand 100201 se déclenche :

Serveur — /var/ossec/etc/ossec.conf

```
<active-response>
  <command>firewall-drop</command>
  <location>local</location>
  <rules_id>100201</rules_id>
  <timeout>180</timeout>
</active-response>
```

Étape 4 — Serveur — valider et appliquer :

```
/var/ossec/bin/wazuh-analysisd -t
systemctl restart wazuh-manager
grep -n "100200" /var/ossec/etc/rules/local_rules.xml
```

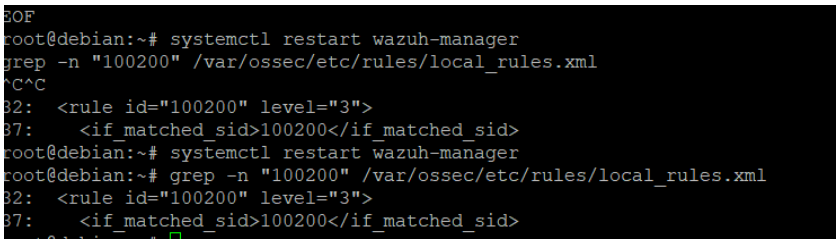


Figure 1 — Manager redémarré, règle 100200 présente dans local_rules.xml

4.3 Avant l'attaque

Une requête unique (`curl`) est décodée par `web-accesslog` et tombe sur la règle native **31108** (niveau 0, « Ignored URLs ») : aucune alerte, comportement normal.

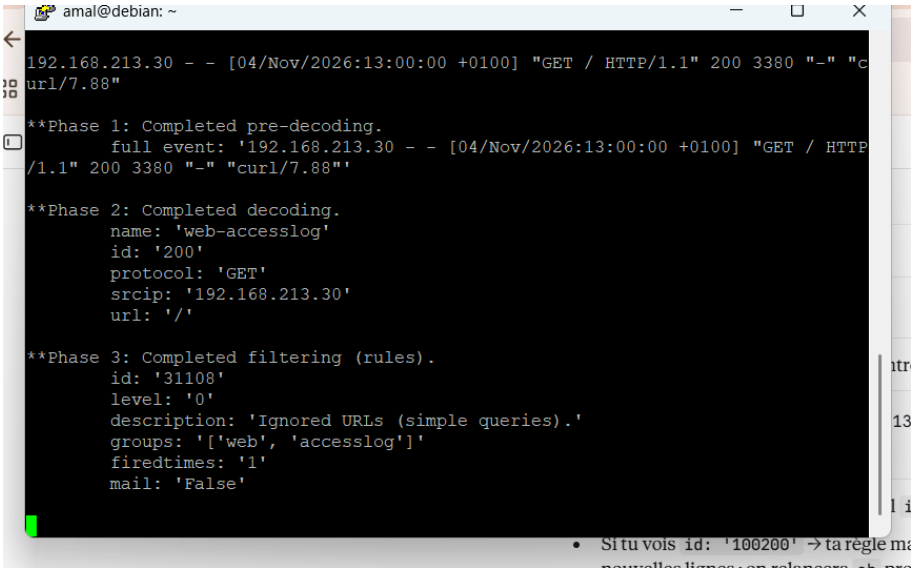


Figure 2 — Avant : wazuh-logtest sur une requête simple → règle 31108, niveau 0

4.4 Pendant l'attaque

```
# Attaquante : 5000 requêtes, 100 en parallèle
ab -n 5000 -c 100 http://192.168.213.134/

# Serveur : suivi des alertes en direct
tail -f /var/ossec/logs/alerts/alerts.log | grep -E "100200|100201"
```

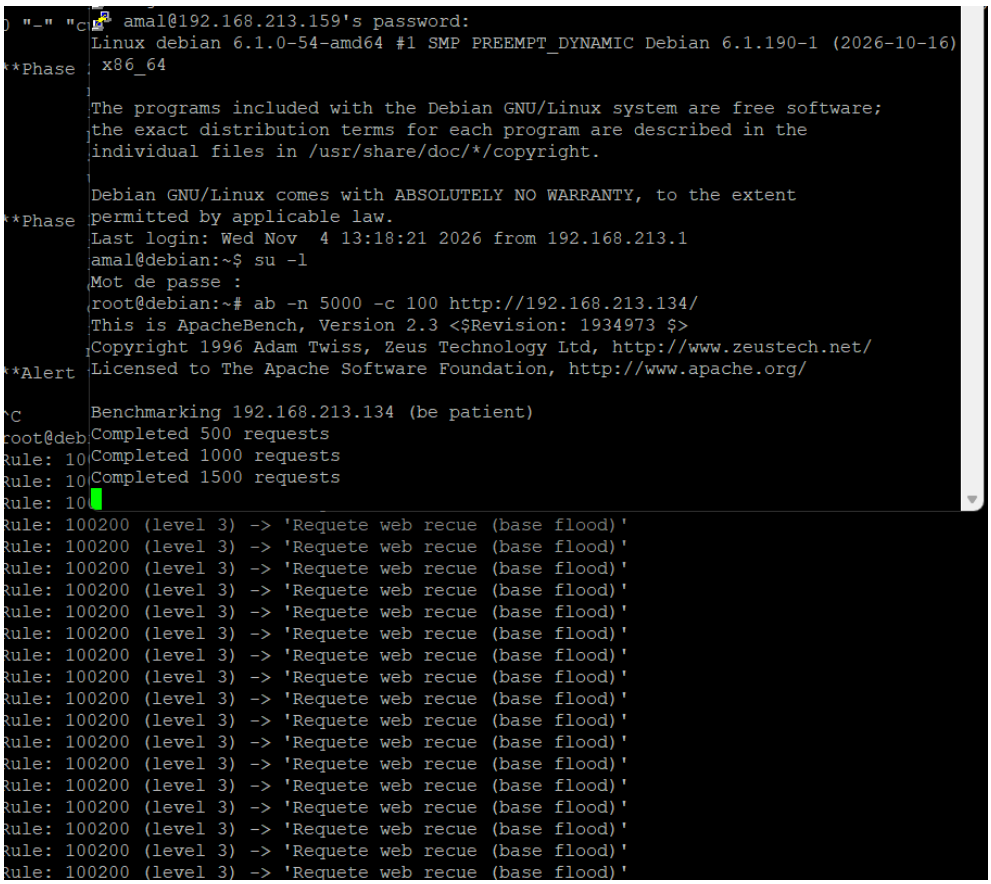


Figure 3 — Pendant : ab en cours, les alertes 100200 défilent sur le serveur

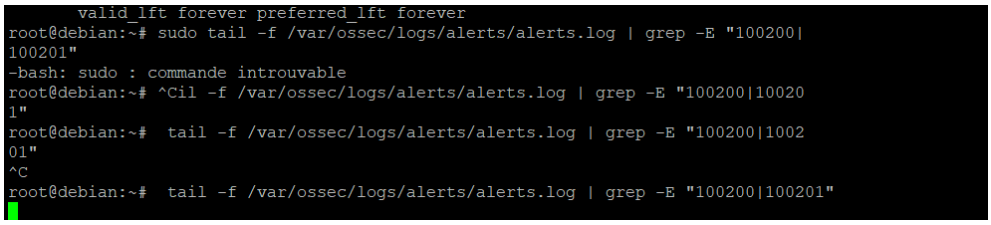


Figure 4 — Commande de surveillance des alertes 100200 / 100201

4.5 Après l'attaque

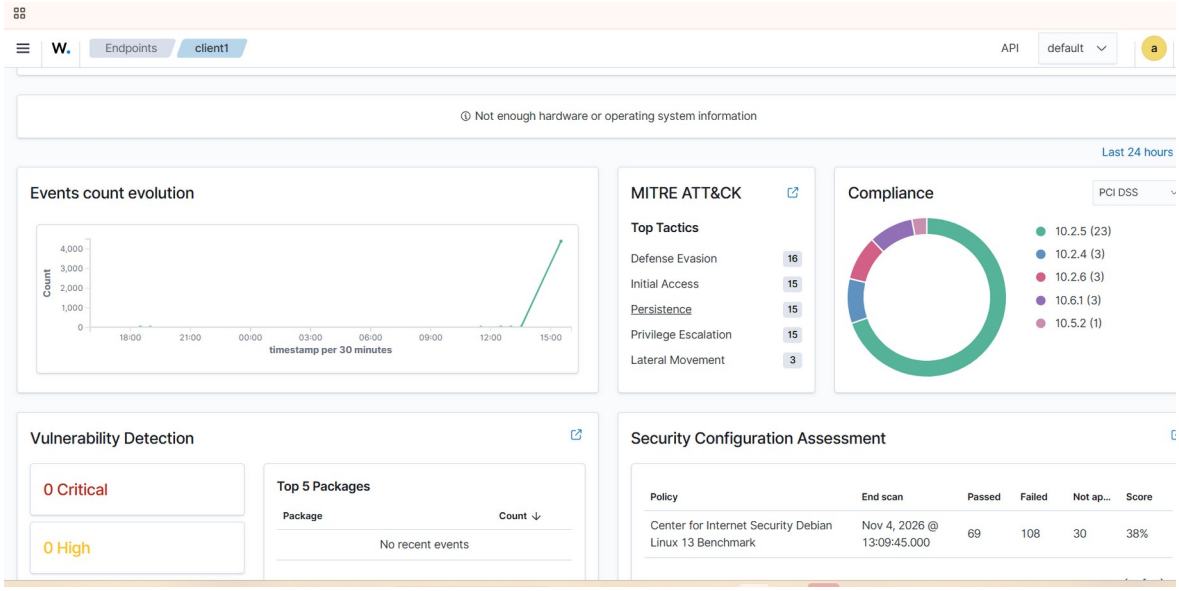


Figure 5 — Après : pic d'environ 4 400 événements sur l'agent client1 vers 15 h

[Capture à insérer]

Alerte 651 « Host Blocked by firewall-drop Active Response » dans le dashboard + sortie de iptables -L INPUT -n sur l'agent montrant le DROP de 192.168.213.159

```
# Agent : preuve du blocage (puis disparition après 180 s)
iptables -L INPUT -n | grep 192.168.213.159
# DROP  all  --  192.168.213.159  0.0.0.0/0
```

4.6 Analyse de l'alerte

Champ	Valeur
rule.id / level	100201 / 10
rule.description	DoS HTTP : plus de 100 requêtes en 10 s depuis 192.168.213.159
rule.mitre.id	T1499 (Impact)
data.srcip	192.168.213.159
agent.name / ip	client1 / 192.168.213.134
rule.frequency / firedtimes	100 / s'incrémente tant que le flood dure

À retenir — Le flood génère des milliers d'alertes 100200 (niveau 3, simple comptage), puis 100201 (niveau 10) une fois le seuil franchi. 100201 déclenche `firewall-drop`, qui ajoute une règle DROP sur l'agent ; la règle **651** confirme le blocage. Après le timeout, l'IP est automatiquement débloquée.

4.7 Limites

- La règle 31108 ne couvre que les requêtes « simples » (ex. `GET /`) : un flood sur d'autres URL ou en POST n'est pas compté.
- Seuil fixe : un pic légitime (NAT d'entreprise, proxy) peut créer un **faux positif** et bloquer des utilisateurs.
- Un **DDoS** réparti sur de nombreuses IP passe sous le seuil `same_source_ip`.
- Les centaines d'alertes 100200 de niveau 3 polluent l'index : passer 100200 en niveau 0 en production.

5. Détection 2 — Injection SQL (MITRE T1190)

5.1 Explication de l'attaque

L'injection SQL consiste à insérer du code SQL dans un paramètre (ici `id`) pour que l'application l'exécute sur sa base de données : lecture de tables, contournement d'authentification… C'est une exploitation d'application exposée (MITRE **T1190 — Exploit Public-Facing Application**).

5.2 Procédure pas à pas

Étape 1 — Aucune configuration supplémentaire : l'agent lit déjà `access.log` (détection 1) et Wazuh fournit les règles natives `0245-web_rules.xml`.

Étape 2 — Attaquante — envoyer une requête contenant un motif SQL :

```
curl -XGET "http://192.168.213.134/users/?id=SELECT+*+FROM+users"
```

Étape 3 — Dashboard — Threat Hunting, filtre `rule.id:31103` sur l'agent 003.

[Capture à insérer]

Pendant : la commande curl et la réponse 404 Not Found d'Apache 2.4.69

[Capture à insérer]

Après : Threat Hunting rule.id:31103 (1 alerte, niveau 7, groupes web / attack / sql_injection)

5.3 Analyse de l'alerte

Champ	Valeur
rule.id / level	31103 / 7
rule.description	SQL injection attempt.
rule.groups	web, accesslog, attack, sql_injection
rule.mitre.id	T1190 (Initial Access)
data.url / data.id	/users/?id=SELECT+*+FROM+users / 404
data.srcip	192.168.213.159

À retenir — La règle 31103 cherche des motifs SQL (`select+`, `union+`, `%20from%20`…) dans l'URL décodée par `web-accesslog`.

La règle **31106** (niveau 12, « A web attack returned code 200 ») ne se déclenche que si le serveur répond **200**. Ici Apache a répondu **404** (page inexistante) : l'attaque est détectée mais a **échoué**, d'où 0 alerte de niveau ≥ 12 dans le dashboard. C'est la bonne lecture : tentative, pas compromission.

5.4 Limites

- Détection par motifs : un encodage différent (%53ELECT, commentaires `/**/`, casse mixte) peut contourner la règle.
- Les requêtes **POST** ne figurent pas dans `access.log` : leur corps n'est pas analysé.
- Faux positifs possibles sur des recherches légitimes contenant « select » ou « union ».
- Pas de réponse active : bloquer sur une seule requête serait trop agressif (amélioration : bloquer sur 31106 ou après N tentatives).

6. Détection 3 — Balayage de ports Nmap (MITRE T1046)

6.1 Explication de l'attaque

Le balayage de ports est une phase de reconnaissance : l'attaquant teste de nombreux ports pour découvrir les services exposés (MITRE **T1046 — Network Service Discovery**). Un scan `-sT` ouvre une connexion TCP complète sur chaque port. Apache ne voit rien : il faut journaliser les paquets au niveau du **pare-feu** de l'agent.

6.2 Procédure pas à pas

Étape 1 — Agent — installer les outils et journaliser chaque SYN entrant :

Agent

```
apt install iptables rsyslog
iptables -I INPUT -p tcp --syn -j LOG --log-prefix "SCAN-DETECT: " --log-level 4
tail -n 20 /var/log/kern.log | grep SCAN-DETECT
```

Étape 2 — Agent — faire lire `kern.log` à l'agent, puis `systemctl restart wazuh-agent` :

Agent — /var/ossec/etc/ossec.conf

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/kern.log</location>
</localfile>
```

Étape 3 — Serveur — règles de corrélation (15 SYN en 5 s depuis la même IP), puis `wazuh-analysisd -t` et redémarrage :

Serveur — /var/ossec/etc/rules/local_rules.xml

```
<group name="local,scan,">
  <rule id="100300" level="3">
    <if_sid>4100</if_sid>
    <match>SCAN-DETECT</match>
    <description>Connexion TCP SYN journalisee par iptables</description>
  </rule>

  <rule id="100301" level="10" frequency="15" timeframe="5">
    <if_matched_sid>100300</if_matched_sid>
    <same_source_ip />
    <description>Scan de ports : plus de 15 SYN en 5 s depuis $(srcip)</description>
    <mitre>
      <id>T1046</id>
    </mitre>
  </rule>
</group>
```

Étape 4 — Attaquante — lancer le scan :

```
nmap -sT -p 1-1000 192.168.213.134
# 22/tcp open ssh, 80/tcp open http, 998 ports fermés (0,24 s)
```

[Capture à insérer]

Avant : recherche rule.id:100301 → « No results match your search criteria »

[Capture à insérer]

Pendant : sortie de nmap -sT -p 1-1000 (22/ssh et 80/http ouverts)

[Capture à insérer]

Après : alerte 100301 dans le dashboard (srcip 192.168.213.159, T1046)

6.3 Analyse de l'alerte

Champ	Valeur
rule.id / level	100301 / 10
rule.description	Scan de ports : plus de 15 SYN en 5 s depuis 192.168.213.159
rule.mitre.id	T1046 (Discovery)
decoder.name	kernel / iptables (SRC, DST, DPT extraits)
data.srcip / data.dstport	192.168.213.159 / port testé

À retenir — Le décodeur iptables natif extrait `srcip` et `dstport` de la ligne kern.log ; la règle native **4100** (groupe firewall) l'attrape d'abord. 100300 en est un enfant (`if_sid 4100`) filtré sur notre préfixe, et 100301 corrèle la fréquence par IP source. 1000 ports scannés en 0,24 s dépassent très largement le seuil.

6.4 Limites

- Un scan **lent** (`nmap -T1`, ou réparti dans le temps) reste sous 15 SYN / 5 s.
- Un scan **UDP** ou furtif (`-sF`, `-sN`, `-sX`) n'envoie pas de SYN et échappe à la règle iptables.
- La règle LOG journalise aussi le trafic légitime (SSH, HTTP) : volume important de kern.log.
- Pas de réponse active : un blocage pourrait toucher un scanner de vulnérabilités autorisé.

7. Analyse des alertes

Règle	Niveau	Description	Déclencheur	MITRE	Réponse
100201	10	DoS HTTP	≥ 100 requêtes simples / 10 s, même IP	T1499	firewall-drop 180 s
31103	7	SQL injection attempt	Motif SQL dans l'URL	T1190	—
31106	12	Web attack returned code 200	31103 + réponse 200 (non atteint : 404)	T1190	—
100301	10	Scan de ports	≥ 15 SYN / 5 s, même IP	T1046	—
651	3	Host Blocked by firewall-drop	Exécution de la réponse active	—	Preuve du blocage

Dans le dashboard, sur 24 h : environ **4 400 événements** concentrés autour de 15 h (flood), **0 alerte de niveau ≥ 12** (aucune attaque n'a réussi) et des tactiques MITRE cohérentes (Initial Access pour l'injection, Impact pour le DoS, Discovery pour le scan).

8. Difficultés rencontrées et solutions

Difficulté	Cause / symptôme	Solution
Pas d'Internet	Réseau isolé : `apt install` impossible	NAT temporaire pour installer les paquets, puis retour en réseau isolé
Confusion agent / serveur	Configs appliquées sur la mauvaise machine	`wazuh-control info | grep type` avant chaque modification
Règle DoS ignorée	`web-accesslog` est un **décodeur**, pas un groupe : `if_group` ne matchait rien	Hériter de la règle native : `if_sid 31108`
Logs non remontés	Agent pas rechargé après l'ajout du `<localfile>`	`systemctl restart wazuh-agent`
Manager planté	Décodeur « iptables » personnalisé inexistant + IDs de règles en double	`wazuh-analysisd -t` systématiquement avant de redémarrer
kern.log absent	Debian utilise journald, pas rsyslog	`apt install rsyslog`
Règle scan sans résultat	La règle native 4100 gagnait (`rule.id:100301` vide)	`if_sid 4100` + niveau 3 sur 100300

9. Limites et améliorations

- **Seuils fixes** → les ajuster après observation du trafic normal (baseline), ou utiliser une liste blanche (CDB list) d'IP de confiance.
- **Contournements** (DDoS distribué, scan lent, encodage SQL) → compléter par un WAF (ModSecurity) et un IDS réseau (Suricata) dont Wazuh lit les logs.
- **Réponse active** → l'étendre au scan et à 31106 avec un timeout progressif, en excluant le serveur Wazuh et l'admin.
- **Bruit** → passer les règles de comptage (100200, 100300) en niveau 0 pour ne garder que les alertes de corrélation.
- **Alerting** → notification e-mail / Slack sur les niveaux ≥ 10.

10. Conclusion

Les trois détections fonctionnent sur le lab : le **DoS HTTP** est détecté par corrélation et bloqué automatiquement (100201 → 651), l'**injection SQL** est reconnue par les règles natives (31103) avec une analyse correcte de son échec (404, pas de 31106), et le **balayage de ports** est rendu visible grâce à la journalisation iptables (100301). Le projet m'a surtout appris à distinguer le rôle de l'agent (collecte) et du serveur (règles), à comprendre la chaîne décodeur → règle → corrélation, et à tester systématiquement (`wazuh-logtest`, `wazuh-analysisd -t`) avant de redémarrer.

11. Sources

- Documentation Wazuh — Rules syntax : https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html
- Documentation Wazuh — Active response : https://documentation.wazuh.com/current/user-manual/capabilities/active-response/
- Documentation Wazuh — wazuh-logtest : https://documentation.wazuh.com/current/user-manual/reference/tools/wazuh-logtest.html
- Règles natives Wazuh : 0245-web_rules.xml (31103, 31106, 31108), règles firewall (4100), 0015-ossec_rules.xml (651)
- MITRE ATT&CK — T1499, T1190, T1046 : https://attack.mitre.org/techniques/
- Code pénal, art. 323-1 et 323-2 : https://www.legifrance.gouv.fr
- Nmap Reference Guide : https://nmap.org/book/man.html — ApacheBench : https://httpd.apache.org/docs/2.4/programs/ab.html
- Sujet « B2 Sécurité Pratique — Détection », A. Lopez, 2026-2027